



UNIVERSIDAD DE LAS FUERZAS ARMADAS

ESPE

Departamento de Ciencias de la Computación.

INTEGRANTES: Martin Palacios
Matias Medina
Leopoldo Bolanos

FECHA: 15 de junio del 2026.

GRUPO: Grupo 7

ASIGNATURA: Metodología de desarrollo de Software.

Modelado UML, Diagrama de casos de uso, Diagrama Puente

DOCENTE: Mgtr. Jenny Ruiz.

NRC: 30693

1.	REQUISITOS FUNCIONALES POR SPRINT	4
2.	SPRINT 1	4
2.1.	DIAGRAMA DE CASOS DE USO	4
2.2.	DIAGRAMA PUENTE.....	9
3.	SPRINT 2	13
3.1.	DIAGRAMA DE CASOS DE USO	13
3.2.	DIAGRAMA PUENTE.....	17
4.	SPRINT 3	22
4.1.	DIAGRAMA DE CASOS DE USO	22
4.2.	DIAGRAMA PUENTE.....	27

1. Listado de Requisitos del Sistema de Gestión de Caja para TAGS y Productos

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
REQ001	El sistema deberá permitir el acceso del administrador mediante usuario y contraseña.	Administrador/ Cajero	Iniciar sesión
REQ002	El sistema deberá permitir la gestión de productos de tienda para agregarlos al cierre de caja.	Administrador/ Cajero	Gestionar productos
REQ003	El sistema deberá permitir el registro de clientes con validación de datos.	Administrador/ Cajero	Registrar cliente
REQ004	El sistema deberá permitir la gestión de servicios TAG.	Administrador/ Cajero	Gestionar servicios TAG
REQ005	El sistema deberá permitir la gestión de servicios variables con concepto y valor.	Administrador/ Cajero	Gestionar servicios variables
REQ006	El sistema deberá permitir consultar y guardar el cierre final de caja en una base de datos no relacional.	Administrador/ Cajero	Cierre de caja
REQ007	El sistema debe permitir ingresar un valor numérico llamado caja chica en el modulo de cierre de caja	Administrador/ Cajero	Caja Chica
REQ008	El sistema deberá permitir buscar por fecha los cierres de caja existentes.	Administrador/ Cajero	Busqueda por Fecha

REQ009	El sistema deberá permitir filtrar por método de pago las ventas que se realice durante una jornada para poder diferenciar el dinero que se tiene en efectivo	Administrador/ Cajero	Metodo de Pago
---------------	---	--------------------------	----------------

1. Requisitos Funcionales por Sprint

Sprint	Requisito funcional
Sprint 1	REQ001 - Acceso al sistema
Sprint 1	REQ002 - Gestión de productos
Sprint 1	REQ003 - Registro de clientes
Sprint 2	REQ004 - Gestión de servicios TAG
Sprint 2	REQ005 - Gestión de servicios variables
Sprint 2	REQ006 - Cierre de caja
Sprint 3	REQ007 – Gestion Caja Chica
Sprint 3	REQ008 - Búsqueda de Cierres de Caja por fecha
Sprint 3	REQ009 – Gestion de Metodos de Pago

2. Sprint 1

2.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml DiagramaCasosDeUso_ProgramaCaja_Sprint1
left to right direction

skinparam usecase {
    BackgroundColor #F5F9FF
    BorderColor #2E6DA4
}

}
```

actor "Administrador / Cajero" as Admin

rectangle "Sistema de Gestión de Caja para TAGS y Productos" {

'=====

' REQ001 - Acceso al Sistema

'=====

usecase "Iniciar sesión" as UC_Login

usecase "Validar campos vacíos\nde usuario y contraseña" as UC_CamposLogin

usecase "Validar credenciales" as UC_ValidarCredenciales

usecase "Acceder al menú principal" as UC_Menu

usecase "Mostrar mensaje de error\nde autenticación" as UC_ErrorLogin

'=====

' REQ002 - Gestión de Productos

'=====

usecase "Gestionar productos\nde tienda" as UC_Productos

usecase "Seleccionar producto" as UC_SeleccionarProducto

usecase "Ajustar cantidad\n(+1 / -1)" as UC_AjustarCantidad

usecase "Guardar productos\nen caja" as UC_GuardarProductos

usecase "Validar existencia\nde productos agregados" as UC_ValidarProductos

usecase "Limpiar productos\ntemporales" as UC_LimpiarProductos

'=====

' REQ003 - Registro de Clientes

'=====

usecase "Registrar cliente" as UC_RegistrarCliente

usecase "Validar campos vacíos\ndel cliente" as UC_CamposCliente

usecase "Validar nombre\nsolo con letras" as UC_ValidarNombre

```

usecase "Validar celular\n10 dígitos y prefijo 09" as UC_ValidarCelular
usecase "Validar correo\nelectrónico" as UC_ValidarCorreo
usecase "Guardar cliente" as UC_GuardarCliente
usecase "Consultar clientes\nregistrados" as UC_CargarClientes
}

```

```

Admin --> UC_Login
Admin --> UC_Productos
Admin --> UC_RegistrarCliente
Admin --> UC_CargarClientes

```

```

'=====

```

```

' Relaciones REQ001

```

```

'=====

```

```

UC_Login ..> UC_CamposLogin : <<include>>

```

```

UC_Login ..> UC_ValidarCredenciales : <<include>>

```

```

UC_Login ..> UC_Menu : <<include>>

```

```

UC_ErrorLogin ..> UC_Login : <<extend>>

```

```

'=====

```

```

' Relaciones REQ002

```

```

'=====

```

```

UC_Productos ..> UC_SeleccionarProducto : <<include>>

```

```

UC_Productos ..> UC_AjustarCantidad : <<include>>

```

```

UC_Productos ..> UC_GuardarProductos : <<include>>

```

```

UC_GuardarProductos ..> UC_ValidarProductos : <<include>>

```

```

UC_LimpiarProductos ..> UC_Productos : <<extend>>

```

```

'=====

```

```

' Relaciones REQ003

```

'=====

UC_RegistrarCliente ..> UC_CamposCliente : <<include>>

UC_RegistrarCliente ..> UC_ValidarNombre : <<include>>

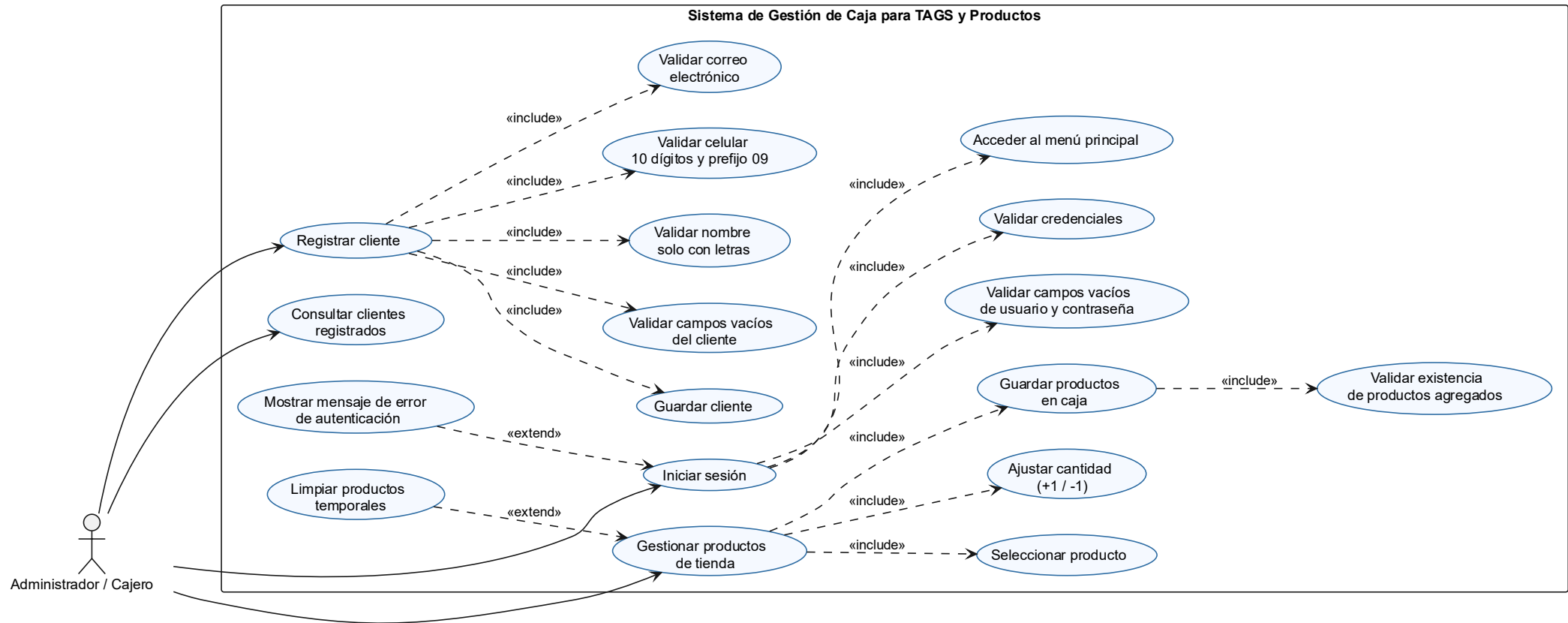
UC_RegistrarCliente ..> UC_ValidarCelular : <<include>>

UC_RegistrarCliente ..> UC_ValidarCorreo : <<include>>

UC_RegistrarCliente ..> UC_GuardarCliente : <<include>>

@enduml

Diagrama de casos de uso en PlantUML



2.2.Diagrama Puente

Código Puente PlantUML

```
@startuml DiagramaPuente_ProgramaCaja_Sprint1
left to right direction

actor "Administrador / Cajero" as Admin

'=====
' REQ001 - Acceso al Sistema
'=====

boundary "LoginView" as UILogin
control "LoginController\n(REQ001)" as CtrlLogin
entity "UserModel" as UserModel
boundary "MainView" as UIMain

Admin --> UILogin : ingresa usuario y contraseña
UILogin --> CtrlLogin : enviar credenciales

CtrlLogin --> CtrlLogin : validar campos vacíos
CtrlLogin --> UserModel : validateCredentials(user, password)
UserModel --> CtrlLogin : retorna resultado

CtrlLogin --> UILogin : mostrar error si los datos son incorrectos
CtrlLogin --> UIMain : abrir menú principal si el acceso es correcto

'=====
' REQ002 - Gestión de Productos
'=====

boundary "SalesPanel" as UISales
control "SalesController\n(REQ002)" as CtrlSales
entity "SalesJournalModel" as SalesModel
entity "CashFlowModel" as CashFlow
```

Admin --> UISales : selecciona producto
 UISales --> CtrlSales : enviar producto seleccionado
 CtrlSales --> SalesModel : addProduct(productName)
 SalesModel --> CtrlSales : retornar cantidades actualizadas
 CtrlSales --> UISales : mostrar resumen temporal

Admin --> UISales : ajusta cantidad
 UISales --> CtrlSales : solicitar +1 / -1
 CtrlSales --> SalesModel : addProduct() / removeProduct()
 SalesModel --> CtrlSales : retornar lista actualizada
 CtrlSales --> UISales : actualizar selector y resumen

Admin --> UISales : limpiar productos temporales
 UISales --> CtrlSales : limpiarProductosTemporales()
 CtrlSales --> SalesModel : clearProducts()
 CtrlSales --> UISales : mostrar resumen vacío

Admin --> UISales : guardar productos en caja
 UISales --> CtrlSales : guardarProductosEnCaja()
 CtrlSales --> SalesModel : getQuantities() y getPrices()
 SalesModel --> CtrlSales : retornar cantidades y precios
 CtrlSales --> CashFlow : addProductSale(producto, cantidad, precio)
 CtrlSales --> SalesModel : clearProducts()
 CtrlSales --> UISales : mostrar confirmación

'=====

' REQ003 - Registro de Clientes

'=====

boundary "ClientPanel" as UIClient
 control "ClientController\n(REQ003)" as CtrlClient
 entity "ClientModel" as ClientModel
 control "ClientRepository" as ClientRepo
 database "MongoDB\nclientes" as MongoClientes

Admin --> UIClient : ingresa datos del cliente

UIClient --> CtrlClient : enviar nombres, celular y correo

CtrlClient --> CtrlClient : validar campos vacíos

CtrlClient --> CtrlClient : validar nombre solo letras

CtrlClient --> CtrlClient : validar celular de 10 dígitos y prefijo 09

CtrlClient --> CtrlClient : validar correo electrónico

CtrlClient --> ClientModel : saveClient(nombres, celular, correo)

ClientModel --> ClientRepo : saveClient(nombres, celular, correo)

ClientRepo --> MongoClientes : insertOne(cliente)

MongoClientes --> ClientRepo : confirmación

ClientRepo --> ClientModel : retorna resultado

ClientModel --> CtrlClient : retorna resultado

CtrlClient --> UIClient : mostrar confirmación o error

Admin --> UIClient : consulta clientes registrados

UIClient --> CtrlClient : cargar lista inicial

CtrlClient --> ClientModel : getCientesList()

ClientModel --> ClientRepo : getCientesList()

ClientRepo --> MongoClientes : find()

MongoClientes --> ClientRepo : documentos encontrados

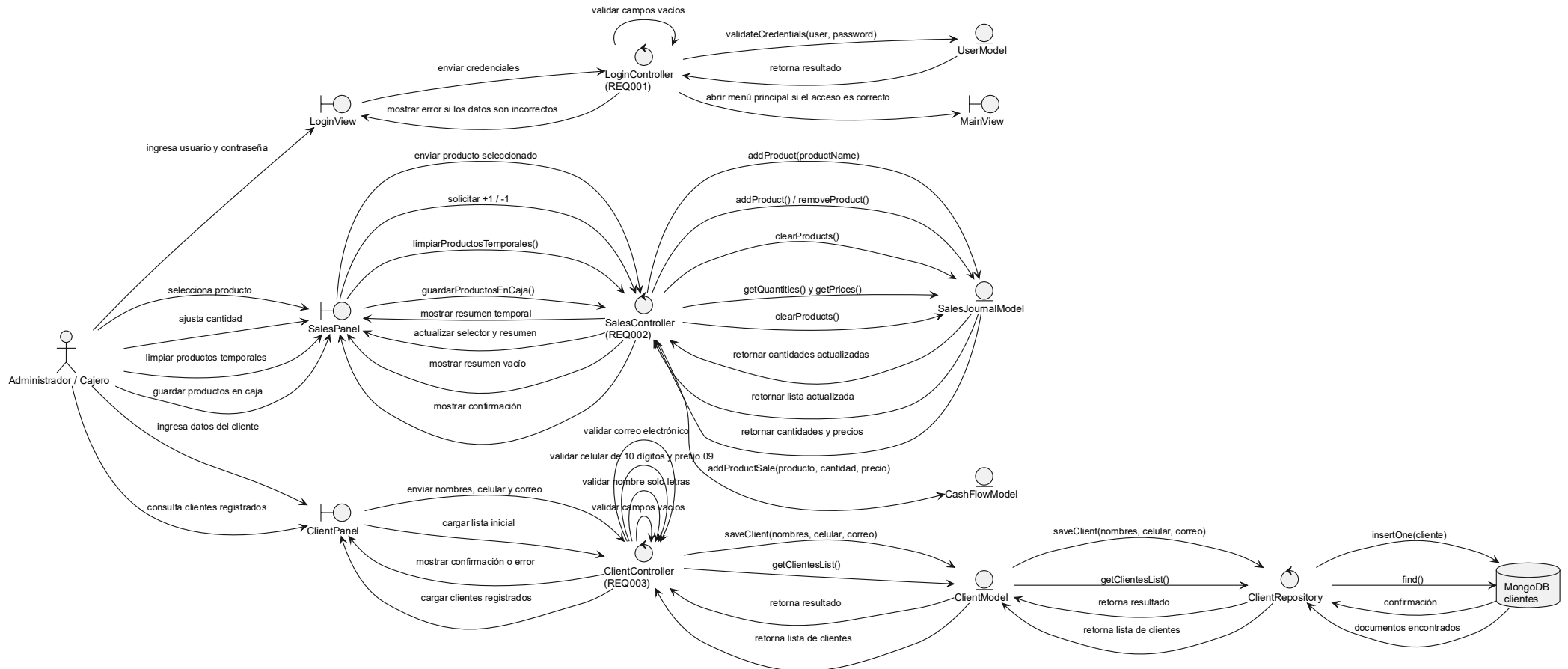
ClientRepo --> ClientModel : retorna lista de clientes

ClientModel --> CtrlClient : retorna lista de clientes

CtrlClient --> UIClient : cargar clientes registrados

@enduml

Diagrama de Puente



3. Sprint 2

3.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml DiagramaCasosDeUso_ProgramaCaja_Sprint2
left to right direction

skinparam usecase {
    BackgroundColor #F5F9FF
    BorderColor #2E6DA4
}

actor "Administrador / Cajero" as Admin

rectangle "Sistema de Gestión de Caja para TAGS y Productos" {

    '=====
    ' REQ004 - Gestión de Servicios TAG
    '=====

    usecase "Gestionar servicios TAG" as UC_Tag
    usecase "Seleccionar servicio TAG" as UC_SeleccionarTag
    usecase "Mostrar precio automático" as UC_PrecioTag
    usecase "Ingresar cantidad" as UC_CantidadTag
    usecase "Validar cantidad\nmayor a cero" as UC_ValidarCantidadTag
    usecase "Agregar servicio TAG\ntemporal" as UC_AgregarTag
    usecase "Guardar servicios TAG\nen caja" as UC_GuardarTag
    usecase "Limpiar servicios TAG\ntemporales" as UC_LimpiarTag

    '=====
    ' REQ005 - Gestión de Servicios Variables
    '=====

    usecase "Gestionar servicios\nvariables" as UC_Variable
```

```

usecase "Ingresar concepto\ny valor" as UC_IngresarVariable
usecase "Validar concepto" as UC_ValidarConcepto
usecase "Validar valor numérico\nmayor a cero" as UC_ValidarValor
usecase "Agregar servicio variable\ntemporal" as UC_AgregarVariable
usecase "Guardar servicios variables\nen caja" as UC_GuardarVariable
usecase "Limpiar servicios variables\ntemporales" as UC_LimpiarVariable

'=====
' REQ006 - Cierre de Caja
'=====

usecase "Consultar reporte final\ndel día" as UC_Reporte
usecase "Calcular subtotales" as UC_Subtotales
usecase "Calcular gran total" as UC_Total
usecase "Validar existencia\nde movimientos" as UC_ValidarMovimientos
usecase "Confirmar cierre\nde jornada" as UC_ConfirmarCierre
usecase "Generar fecha y hora\nactual" as UC_FechaHora
usecase "Guardar jornada\nen MongoDB" as UC_GuardarJornada
usecase "Reiniciar caja actual" as UC_ReiniciarCaja
}

Admin --> UC_Tag
Admin --> UC_Variable
Admin --> UC_Reporte
Admin --> UC_GuardarJornada

'=====
' Relaciones REQ004
'=====

UC_Tag ..> UC_SeleccionarTag : <<include>>
UC_Tag ..> UC_PrecioTag : <<include>>
UC_Tag ..> UC_CantidadTag : <<include>>
UC_AgregarTag ..> UC_ValidarCantidadTag : <<include>>
UC_Tag ..> UC_AgregarTag : <<include>>

```

UC_Tag ..> UC_GuardarTag : <<include>>

UC_LimpiarTag ..> UC_Tag : <<extend>>

'=====

' Relaciones REQ005

'=====

UC_Variable ..> UC_IngresarVariable : <<include>>

UC_Variable ..> UC_ValidarConcepto : <<include>>

UC_Variable ..> UC_ValidarValor : <<include>>

UC_Variable ..> UC_AgregarVariable : <<include>>

UC_Variable ..> UC_GuardarVariable : <<include>>

UC_LimpiarVariable ..> UC_Variable : <<extend>>

'=====

' Relaciones REQ006

'=====

UC_Reporte ..> UC_Subtotales : <<include>>

UC_Reporte ..> UC_Total : <<include>>

UC_GuardarJornada ..> UC_ValidarMovimientos : <<include>>

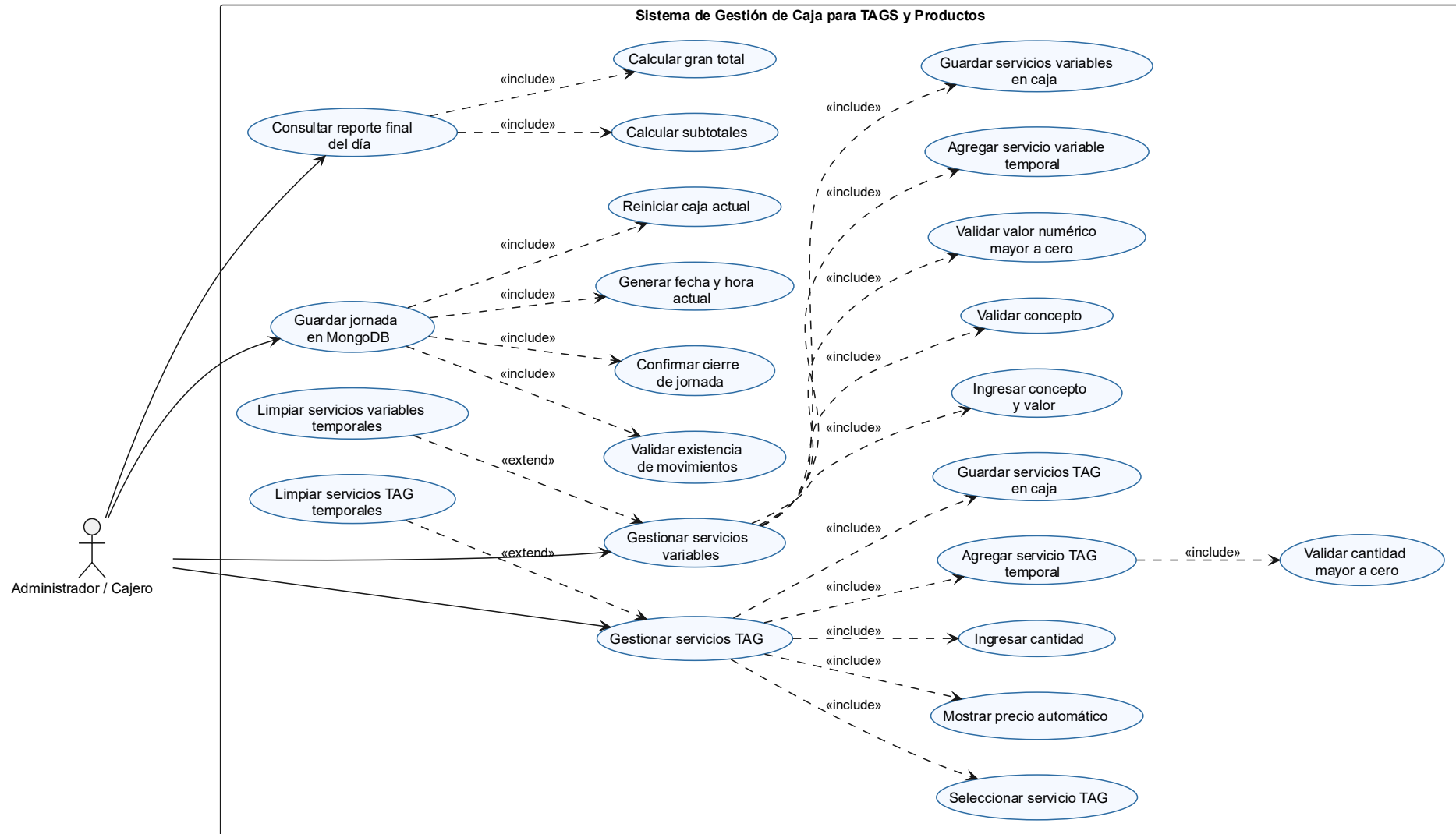
UC_GuardarJornada ..> UC_ConfirmarCierre : <<include>>

UC_GuardarJornada ..> UC_FechaHora : <<include>>

UC_GuardarJornada ..> UC_ReiniciarCaja : <<include>>

@enduml

Diagrama de casos de uso en PlantUML



3.2. Diagrama Puente

Codigo Puente PlantUML

```
@startuml DiagramaPuente_ProgramaCaja_Sprint2
left to right direction

actor "Administrador / Cajero" as Admin

'=====
' REQ004 - Gestión de Servicios TAG
'=====

boundary "TagServicePanel" as UITag
control "TagServiceController\n(REQ004)" as CtrlTag
entity "Catálogo TAG" as CatalogoTag
entity "Servicios TAG\nTemporales" as TempTag
entity "CashFlowModel" as CashFlow

Admin --> UITag : selecciona servicio TAG
UITag --> CtrlTag : obtener servicio seleccionado
CtrlTag --> CatalogoTag : consultar precio
CatalogoTag --> CtrlTag : retorna precio
CtrlTag --> UITag : mostrar precio automático

Admin --> UITag : ingresa cantidad y agrega servicio
UITag --> CtrlTag : agregarServicioTemporal()
CtrlTag --> CtrlTag : validar servicio y cantidad
CtrlTag --> TempTag : agregar servicio temporal
CtrlTag --> UITag : actualizar resumen temporal

Admin --> UITag : limpiar servicios TAG temporales
UITag --> CtrlTag : limpiarServiciosTemporales()
CtrlTag --> TempTag : clear()
```

CtrlTag --> UITag : mostrar resumen vacío

Admin --> UITag : guardar servicios TAG en caja

UITag --> CtrlTag : guardarServiciosEnCaja()

CtrlTag --> CashFlow : addTagServiceSale(servicio, cantidad, precio)

CtrlTag --> TempTag : clear()

CtrlTag --> UITag : mostrar confirmación

'=====

' REQ005 - Gestión de Servicios Variables

'=====

boundary "VariableServicePanel" as UIVar

control "VariableServiceController\n(REQ005)" as CtrlVar

entity "Servicios Variables\nTemporales" as TempVar

Admin --> UIVar : ingresa concepto y valor

UIVar --> CtrlVar : agregarServicioTemporal()

CtrlVar --> CtrlVar : validar concepto

CtrlVar --> CtrlVar : validar valor numérico

CtrlVar --> TempVar : agregar servicio temporal

CtrlVar --> UIVar : actualizar resumen temporal

Admin --> UIVar : limpiar servicios variables temporales

UIVar --> CtrlVar : limpiarServiciosTemporales()

CtrlVar --> TempVar : clear()

CtrlVar --> UIVar : mostrar resumen vacío

Admin --> UIVar : guardar servicios variables en caja

UIVar --> CtrlVar : guardarServiciosEnCaja()

CtrlVar --> CashFlow : addVariableServiceSale(concepto, valor)

CtrlVar --> TempVar : clear()

CtrlVar --> UIVar : mostrar confirmación

```
'=====
```

```
' REQ006 - Cierre de Caja
```

```
'=====
```

```
boundary "CashFlowPanel" as UICash
```

```
control "CashFlowController\n(REQ006)" as CtrlCash
```

```
control "CashFlowRepository" as CashRepo
```

```
database "MongoDB\ncierres_caja" as MongoCierres
```

```
Admin --> UICash : consulta reporte final
```

```
UICash --> CtrlCash : actualizarReporte()
```

```
CtrlCash --> CashFlow : getFinalReport()
```

```
CashFlow --> CtrlCash : retorna reporte
```

```
CtrlCash --> UICash : mostrar reporte final
```

```
Admin --> UICash : guardar jornada
```

```
UICash --> CtrlCash : guardarJornada()
```

```
CtrlCash --> CashFlow : hasMovements()
```

```
CashFlow --> CtrlCash : retorna resultado
```

```
CtrlCash --> UICash : solicitar confirmación
```

```
Admin --> UICash : confirma cierre
```

```
CtrlCash --> CashFlow : saveJournalAndReset()
```

```
CashFlow --> CashFlow : generar fecha y hora actual
```

```
CashFlow --> CashFlow : calcular subtotales y gran total
```

```
CashFlow --> CashRepo : saveCashClosure(cierreCaja)
```

```
CashRepo --> MongoCierres : insertOne(cierreCaja)
```

```
MongoCierres --> CashRepo : confirmación
```

```
CashRepo --> CashFlow : retorna resultado
```

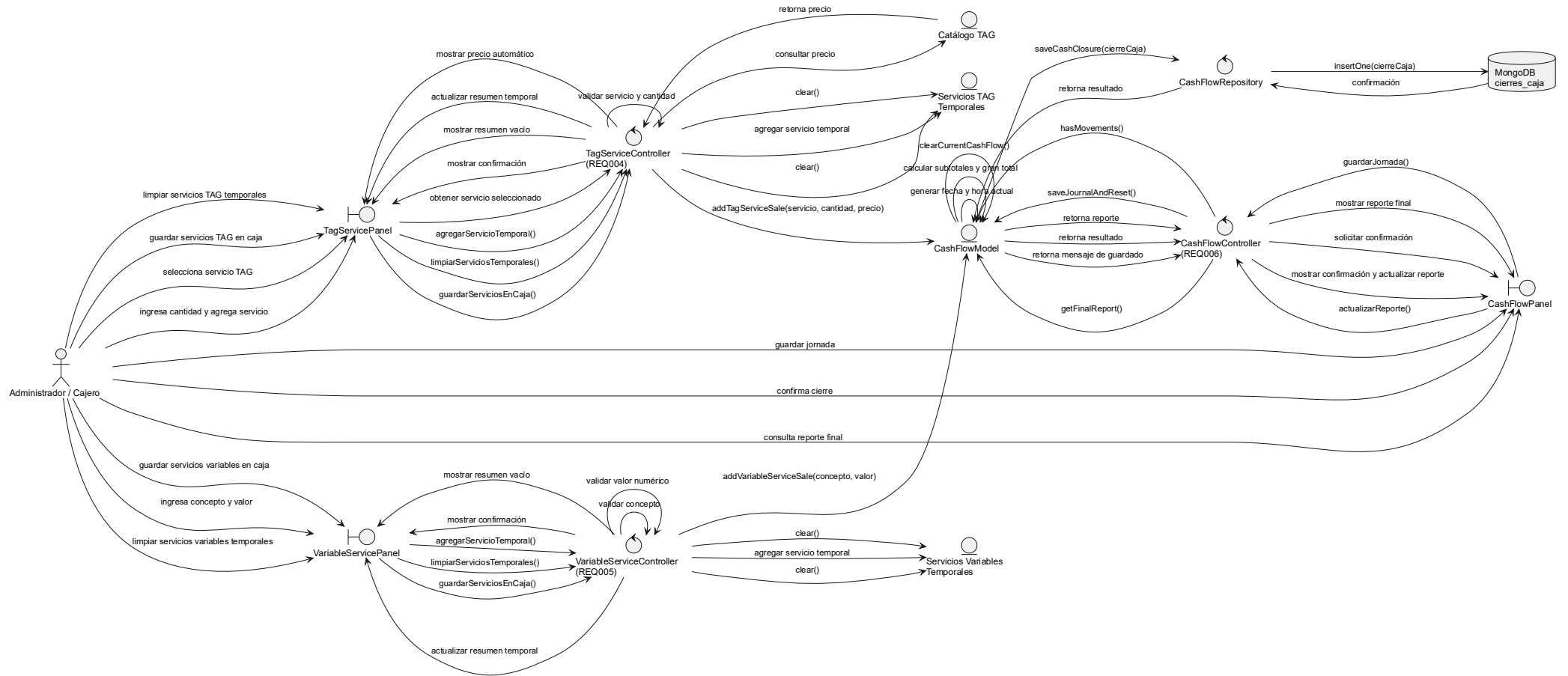
```
CashFlow --> CashFlow : clearCurrentCashFlow()
```

```
CashFlow --> CtrlCash : retorna mensaje de guardado
```

CtrlCash --> UICash : mostrar confirmación y actualizar reporte

@enduml

Diagrama Puente



4. Sprint 3

4.1. Diagrama de Casos de uso

Código en PlantUML

```
@startuml DiagramaCasosDeUso_ProgramaCaja_Sprint3
```

```
left to right direction
```

```
skinparam usecase {
```

```
    BackgroundColor #F5F9FF
```

```
    BorderColor #2E6DA4
```

```
}
```

```
actor "Administrador / Cajero" as Admin
```

```
rectangle "Sistema de Gestión de Caja para TAGS y Productos" {
```

```
    '=====
```

```
    ' REQ007 - Gestión de Caja Chica
```

```
    '=====
```

```
    usecase "Gestionar caja chica" as UC_CajaChica
```

```
    usecase "Visualizar caja\ncnica actual" as UC_VerCajaActual
```

```
    usecase "Ingresar caja chica\ninicial" as UC_CajaInicial
```

```
    usecase "Ingresar caja chica\npara mañana" as UC_CajaManana
```

```
    usecase "Validar valor\nnumérico" as UC_ValidarCaja
```

```
    usecase "Guardar caja chica" as UC_GuardarCaja
```

```
    usecase "Guardar caja chica\nen cierre" as UC_CajaEnCierre
```

```
    '=====
```

```
    ' REQ008 - Búsqueda de Cierres por Fecha
```

'=====

usecase "Consultar historial\nde cierres" as UC_Historial

usecase "Buscar cierre\npor fecha" as UC_BuscarFecha

usecase "Validar fecha\ndd/MM/yyyy" as UC_ValidarFecha

usecase "Mostrar cierres\nencontrados" as UC_MostrarCierres

usecase "Mostrar historial\ncompleto" as UC_MostrarTodos

'=====

' REQ009 - Gestión de Métodos de Pago

'=====

usecase "Gestionar método\nde pago" as UC_MetodoPago

usecase "Visualizar venta\nactual" as UC_VerVenta

usecase "Seleccionar método\nde pago" as UC_SeleccionarMetodo

usecase "Registrar pago\nen efectivo" as UC_PagoEfectivo

usecase "Registrar pago por\ntransferencia" as UC_PagoTransferencia

usecase "Registrar pago\ncon tarjeta" as UC_PagoTarjeta

usecase "Registrar pago\nmixto" as UC_PagoMixto

usecase "Validar montos\ndel pago mixto" as UC_ValidarMixto

usecase "Confirmar pago" as UC_ConfirmarPago

usecase "Enviar venta\na caja" as UC_EnviarCaja

usecase "Limpiar venta\nactual" as UC_LimpiarVenta

}

Admin --> UC_CajaChica

Admin --> UC_Historial

Admin --> UC_MetodoPago

'=====

' Relaciones REQ007

'=====

UC_CajaChica ..> UC_VerCajaActual : <<include>>

UC_CajaChica ..> UC_CajaManana : <<include>>

UC_GuardarCaja ..> UC_ValidarCaja : <<include>>

UC_CajaChica ..> UC_GuardarCaja : <<include>>

UC_CajaChica ..> UC_CajaEnCierre : <<include>>

UC_CajaInicial ..> UC_CajaChica : <<extend>>

'=====

' Relaciones REQ008

'=====

UC_Historial ..> UC_BuscarFecha : <<include>>

UC_BuscarFecha ..> UC_ValidarFecha : <<include>>

UC_BuscarFecha ..> UC_MostrarCierres : <<include>>

UC_Historial ..> UC_MostrarTodos : <<include>>

'=====

' Relaciones REQ009

'=====

UC_MetodoPago ..> UC_VerVenta : <<include>>

UC_MetodoPago ..> UC_SeleccionarMetodo : <<include>>

UC_MetodoPago ..> UC_ConfirmarPago : <<include>>

UC_ConfirmarPago ..> UC_EnviarCaja : <<include>>

UC_LimpiarVenta ..> UC_MetodoPago : <<extend>>

UC_PagoEfectivo ..> UC_SeleccionarMetodo : <<extend>>

UC_PagoTransferencia ..> UC_SeleccionarMetodo : <<extend>>

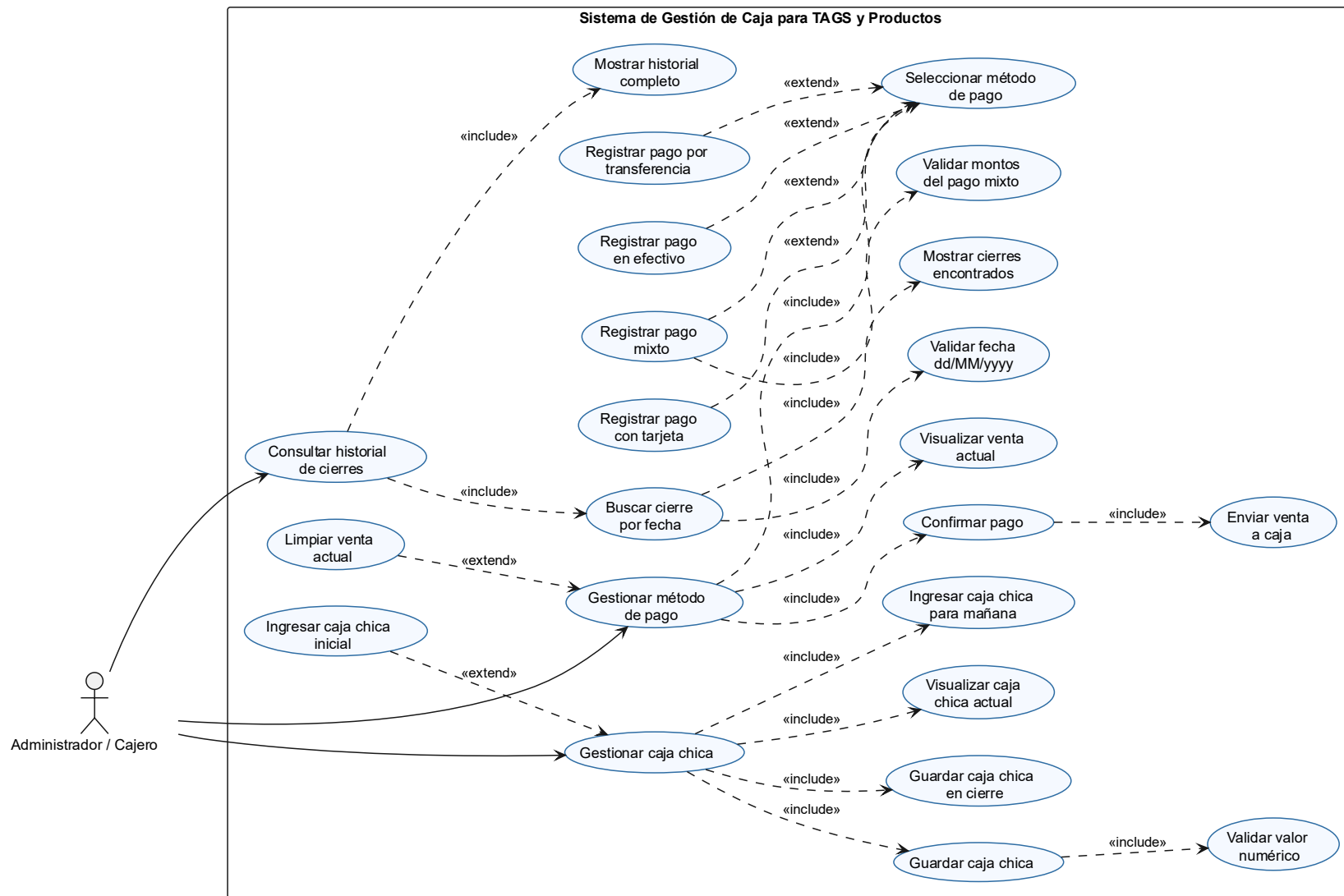
UC_PagoTarjeta ..> UC_SeleccionarMetodo : <<extend>>

UC_PagoMixto ..> UC_SeleccionarMetodo : <<extend>>

UC_PagoMixto ..> UC_ValidarMixto : <<include>>

@enduml

Diagrama de casos de uso en PlantUML



4.2.Diagrama Puente

Código Puente PlantUML

```
@startuml DiagramaPuente_ProgramaCaja_Sprint3_Simplificado

left to right direction

actor "Administrador / Cajero" as Admin

'=====
' REQ007 - Caja Chica
'=====

boundary "CashFlowPanel" as CashFlowPanel
control "CashFlowController\n(REQ007)" as CashFlowController
entity "CashFlowModel" as CashFlowModel
control "CashFlowRepository" as CashFlowRepository
database "MongoDB\ncierres_caja" as MongoDB

Admin --> CashFlowPanel : consulta cierre de caja
CashFlowPanel --> CashFlowController : solicitar reporte y caja chica
CashFlowController --> CashFlowModel : obtener datos de cierre
CashFlowModel --> CashFlowRepository : consultar último cierre
CashFlowRepository --> MongoDB : buscar cierre guardado
MongoDB --> CashFlowRepository : retorna datos
CashFlowRepository --> CashFlowModel : retorna caja chica actual
CashFlowModel --> CashFlowController : retorna reporte
CashFlowController --> CashFlowPanel : mostrar reporte y caja chica

Admin --> CashFlowPanel : guarda jornada
CashFlowPanel --> CashFlowController : guardar jornada
CashFlowController --> CashFlowModel : registrar cierre con caja chica
CashFlowModel --> CashFlowRepository : guardar cierre
```

CashFlowRepository --> MongoDB : insertOne(cierreCaja)

MongoDB --> CashFlowRepository : confirmación

CashFlowRepository --> CashFlowModel : resultado

CashFlowModel --> CashFlowController : mensaje de guardado

CashFlowController --> CashFlowPanel : mostrar confirmación

'=====

' REQ008 - Historial por Fecha

'=====

boundary "CashHistoryPanel" as CashHistoryPanel

control "CashHistoryController\n(REQ008)" as CashHistoryController

entity "CashHistoryModel" as CashHistoryModel

Admin --> CashHistoryPanel : busca cierres por fecha

CashHistoryPanel --> CashHistoryController : enviar fecha

CashHistoryController --> CashHistoryModel : buscar historial

CashHistoryModel --> CashFlowRepository : consultar cierres por fecha

CashFlowRepository --> MongoDB : buscar cierres

MongoDB --> CashFlowRepository : retorna coincidencias

CashFlowRepository --> CashHistoryModel : retorna resultados

CashHistoryModel --> CashHistoryController : historial encontrado

CashHistoryController --> CashHistoryPanel : mostrar resultados

Admin --> CashHistoryPanel : solicita mostrar todos

CashHistoryPanel --> CashHistoryController : mostrar historial completo

CashHistoryController --> CashHistoryModel : obtener todos los cierres

CashHistoryModel --> CashFlowRepository : consultar todos

CashFlowRepository --> MongoDB : find()

MongoDB --> CashFlowRepository : retorna documentos

CashFlowRepository --> CashHistoryModel : retorna historial

CashHistoryModel --> CashHistoryController : historial completo

CashHistoryController --> CashHistoryPanel : mostrar historial completo

```
'=====
```

```
' REQ009 - Método de Pago
```

```
'=====
```

```
boundary "PaymentPanel" as PaymentPanel
```

```
control "PaymentController\n(REQ009)" as PaymentController
```

```
entity "CurrentSaleModel" as CurrentSaleModel
```

```
entity "PaymentDetail" as PaymentDetail
```

```
Admin --> PaymentPanel : revisa venta actual
```

```
PaymentPanel --> PaymentController : actualizar venta
```

```
PaymentController --> CurrentSaleModel : obtener venta actual
```

```
CurrentSaleModel --> PaymentController : retorna resumen
```

```
PaymentController --> PaymentPanel : mostrar resumen
```

```
Admin --> PaymentPanel : confirma método de pago
```

```
PaymentPanel --> PaymentController : enviar método y valores
```

```
PaymentController --> CurrentSaleModel : validar venta actual
```

```
PaymentController --> PaymentDetail : construir detalle de pago
```

```
PaymentDetail --> PaymentController : retorna detalle
```

```
PaymentController --> CashFlowModel : registrar venta en caja
```

```
CashFlowModel --> PaymentController : confirmación
```

```
PaymentController --> CurrentSaleModel : limpiar venta actual
```

```
PaymentController --> PaymentPanel : mostrar confirmación
```

```
@enduml
```

Diagrama de casos de uso en PlantUML

